

Fecha de Entrega: 25/05/26

Interacción Humano Computadora – 2026A



Hands-On 4: Actividad Integradora

Centro universitario de ciencias exactas e ingenierías

Maestro

Jose Antonio Aviña Mendez

Estudiantes

Jaime Ramos Avila, Ingeniero en Computación

Código de Estudiante: 2997985

Humberto de Jesús Peña Dueñas, Ingeniero en Computación

Código de Estudiante: 223992329

Índice

●	Introducción al Modelado Matemático.....	3
●	Transformaciones Matemáticas en el Entorno 3D.....	3
○	Traslación.....	3
○	Rotación.....	4
○	Rebote (Cinemática con Gravedad).....	4
○	Movimiento Senoidal.....	5
○	Trayectoria Ondulada.....	5
○	Órbita Circular.....	6
●	Proyecto Integrador	7
●	Conclusiones.....	7
●	Referencias Bibliográficas.....	8

- **Introducción al Modelado Matemático**

Las funciones matemáticas son importantes para la construcción de valores deterministas dentro de un sistema. Para los gráficos, sabemos que esta función debe de intervenir con respecto al valor del tiempo, por lo que en este momento lo que vamos a abordar son propiedades muy importantes para considerar los elementos vitales que nos servirán para desarrollar poco a poco dichas funciones. Dentro de este recorrido, explicaremos las técnicas matemáticas que nos pueden apoyar para entender cada cuerpo, cada movimiento y dentro de un rango considerable de movimiento. Nos relacionaremos con los conceptos de la animación y pondremos a prueba mediante código y una interacción directa con nuestras librerías de movimiento matemático.

- **¿Qué Movimientos modelamos?**

Existen diferentes modelos dependiendo de la funcionalidad que nuestro cuerpo va a tener dentro de un determinado tiempo. El cuerpo se le llama a todo objeto que será nuestro centro para brindar animación, el cual se moverá y la **función** determina cómo es que se va a mover, esto se refiere si es lineal, senoidal o realiza rebotes.

Transformaciones Matemáticas en el Entorno 3D

A continuación, se analizan matemáticamente cada una de las transformaciones solicitadas y se asocian directamente con las líneas de código fuente en Raylib/C++ utilizadas para su implementación en el espacio tridimensional. Para cada movimiento existe una transformación que es la representación del comportamiento que tendrá nuestro objeto según los parámetros establecidos, esto quiere decir que si hablamos de una trayectoria representada vamos a poner el código que represente dicho cambio que existirá conforme al tiempo.

Traslación

La traslación es una transformación afín que desplaza cada punto de una figura o espacio a la misma distancia en una dirección dada. Matemáticamente, dado un punto $P = (x, y, z)$ y un vector de traslación $T = (tx, ty, tz)$, el nuevo punto P' se calcula como: $P' = P + T = (x + tx, y + ty, z + tz)$

En nuestra aplicación, la traslación pura sobre un eje se observa al actualizar la posición de un objeto desplazándose constantemente a lo largo de su coordenada x basándose en el transcurso del tiempo, aplicando una operación de módulo para que reinicie su recorrido y se mantenga en un ciclo finito.

Líneas de código implementadas:

```
C/C++
float t = (float)GetTime();
// Traslación en el eje X usando una función de tiempo y
módulo
float trayectoriaX = -2.5f + fmodf(t * 1.5f, 5.0f);
```

Rotación

La rotación es el movimiento de cambio de orientación de un cuerpo de forma que, dado un punto cualquiera del mismo, este permanece a una distancia constante de un punto fijo (centro de rotación) o eje. En 3D, una rotación en torno al eje Y por un ángulo θ se define por la matriz de rotación, resultando en las ecuaciones paramétricas:

- $x' = x \cos(\theta) - z \sin(\theta)$
- $z' = x \sin(\theta) + z \cos(\theta)$

En el código, la rotación global de la escena es manejada de forma nativa por el sistema de cámaras orbitales de la librería, la cual recalcula continuamente los vectores directores de la cámara alrededor del punto objetivo (*target*).

Líneas de código implementadas:

```
C/C++
Camera3D camera = { 0 };
camera.position = { 0.0f, 18.0f, 20.0f };
camera.target = { 0.0f, 0.0f, 0.0f };
// Aplicación de la rotación automática sobre el objetivo
UpdateCamera(&camera, CAMERA_ORBITAL);
```

Rebote (Cinemática con Gravedad)

El rebote simula la física clásica de un cuerpo en caída libre bajo la influencia de la gravedad, experimentando una colisión elástica (o semi-elástica) al impactar contra el suelo. Se rige por las ecuaciones del movimiento uniformemente acelerado:

- $v_f = v_0 + a(\Delta t)$
- $y_f = y_0 + v_0(\Delta t) + 1/2 a(\Delta t)^2$

Al tocar un límite definido (el piso), la dirección de la velocidad se invierte y se multiplica por un coeficiente de restitución (< 1) para simular la pérdida de energía cinética.

Líneas de código implementadas:

```
C/C++
float reboteY = 4.0f;
float velocidadY = 0.0f;
float gravedad = -0.015f;

// Actualización física en cada frame
velocidadY += gravedad;
reboteY += velocidadY;

// Evaluación de límite y rebote (inversión de velocidad con
amortiguamiento)
if (reboteY < 1.0f) {
    reboteY = 1.0f;
    velocidadY *= -0.90f; // Coeficiente de restitución
}
```

Movimiento Senoidal

El movimiento senoidal describe una oscilación periódica suave, modelada matemáticamente a través de las funciones trigonométricas (seno o coseno). Su ecuación general con respecto al tiempo t es: $y(t) = A \sin(\omega t) + C$ Donde A es la amplitud de la onda, ω es la frecuencia angular, y C es el desplazamiento en el eje y .

Líneas de código implementadas:

```
C/C++
float t = (float)GetTime();
// A = 1.5f, w = 2.0f, C = 2.0f
float senoY = 2.0f + sinf(t * 2.0f) * 1.5f;
```

Trayectoria Ondulada

Una trayectoria paramétrica compleja surge al combinar múltiples transformaciones simultáneas en diferentes ejes. En este modelo, el cuerpo experimenta una traslación lineal en el

eje X y una oscilación senoidal simultánea en el eje Z , generando un patrón de onda en el espacio 3D. Ecuaciones paramétricas:

- $x(t) = v * t$
- $z(t) = A \sin(\omega z * t)$

Líneas de código implementadas:

```
C/C++  
float t = (float)GetTime();  
// Combinación de traslación limitada y oscilación  
float trayectoriaX = -2.5f + fmodf(t * 1.5f, 5.0f);  
float trayectoriaZ = sinf(t * 3.0f) * 1.2f;
```

Órbita Circular

Una órbita perfecta en un plano 2D inmerso en 3D (el plano XZ en este caso) se describe utilizando las coordenadas polares convertidas a coordenadas cartesianas. La distancia al centro es el radio r , y el ángulo respecto al tiempo es t .

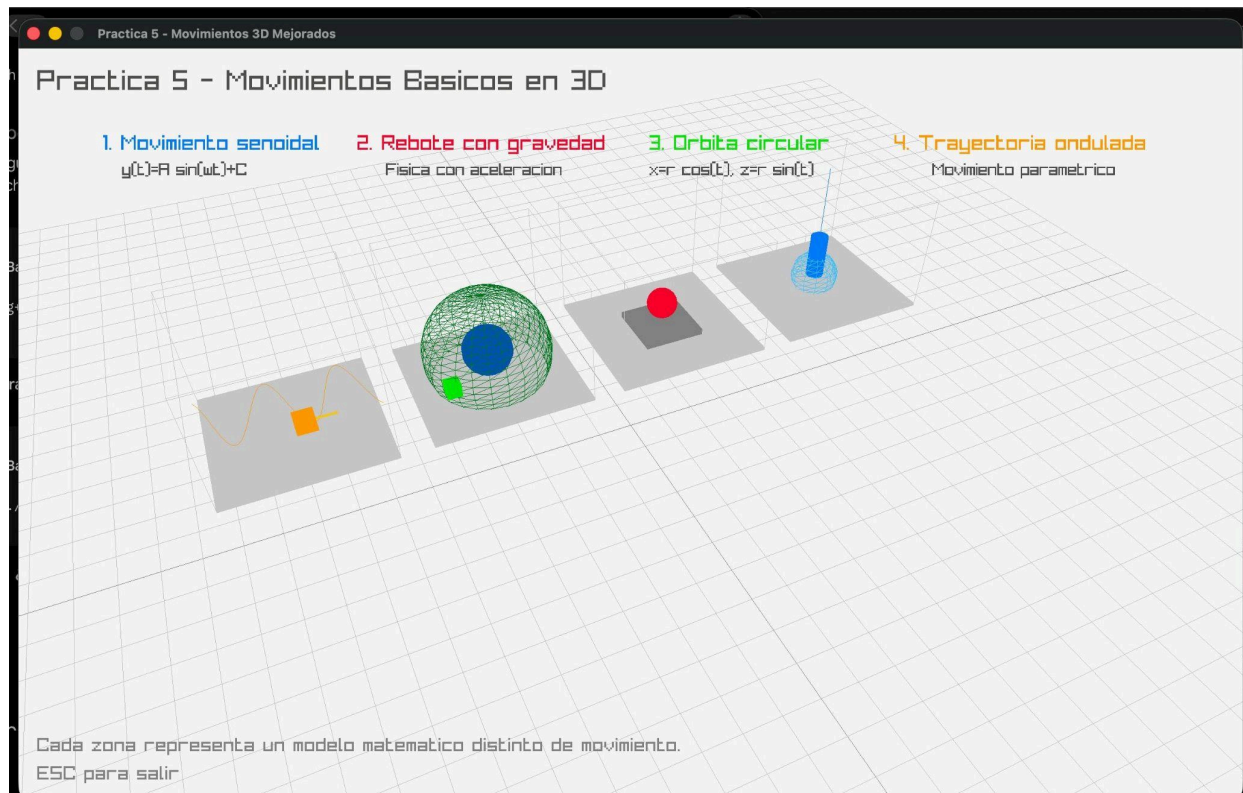
- $x(t) = r \cos(t)$
- $z(t) = r \sin(t)$ Esto crea un movimiento circular uniforme alrededor del origen local asignado.

Líneas de código implementadas:

```
C/C++  
float t = (float)GetTime();  
float radio = 2.0f;  
  
// Ecuaciones paramétricas de la circunferencia  
float orbitaX = cosf(t) * radio;  
float orbitaZ = sinf(t) * radio;
```

Proyecto Integrador

Buscamos implementar un entorno donde se pongan a prueba estos cuatro modelos matemáticos y los visualizamos de manera que interactuen con cuerpos, el usuario pueda revisar y entender cómo es que funcionan en la naturalidad y comportamiento de objetos estas propiedades y su distinción al incluir que en el proyecto se divida cada uno de los movimientos.



Conclusión: La importancia del modelado para la animación

Prácticamente, llegamos a la idea en la cual tener un modelo matemático para definir el comportamiento de un cuerpo que también está por construirse es esencial para saber cuáles serán sus rangos, capacidades y funciones. Es importante saber que cada parámetro en el desarrollo de un sistema de animación computacional es abstracto para conocer qué es lo que representa para el cuerpo que se está digitalizando, al igual que entender bien cómo aplicar estos conceptos de rotación, traslación y cambio con respecto al tiempo ayudan a la elaboración de un entendimiento sólido del desplazamiento de particular, así como puede involucrar temas muy importantes como la colisión de dos cuerpos y según sus características, comprender qué es lo que sucederá con sus valores de cambio.

Referencias Bibliográficas

- Casanova, M. (2018). *Matemáticas para la Computación Gráfica y el Diseño de Juegos*. Madrid, España: Ediciones Universitarias.
- Eberly, D. H. (2015). *3D game engine architecture: engineering real-time applications with C++*. CRC Press.
- Sanjurjo, R. (2020). *Raylib Programming: Videogames in C/C++*. Recuperado de la documentación oficial de Raylib: <https://www.raylib.com/>
- Cordova, D. G., Flores, E. N., García, R. R., & Salvador, J. C. R. (s/f). *Modelación matemática y computacional: la respuesta al anhelo ancestral de predecir a la naturaleza*. Ciencia UNAM. Recuperado el 25 de mayo de 2026, de https://ciencia.unam.mx/leer/115/Modelacion_matematica_y_computacional_la_respuesta_al_anhelo_ancestral_de_predecir_a_la_naturaleza
- Verma, N. (2021, agosto 23). *Taylor series- for dummies, by a dummy*. Medium. <https://medium.com/@nisheetsun/taylor-series-for-dummies-by-a-dummy-6b3a703ae084>